A miniature DynamoDB with replication

DynamoMINI

Ronit Jalihal (2023101028)

Kushagra Trivedi (2023101048)

Prasoon Dev (2023111014)

30th November, 2025

INTRODUCTION

- DynamoMINI is a toy version of Amazon's Dynamo, a highly available key-value storage system designed to provide scalability and low latency.

- DynamoDB was designed as a decentralized, key-value store that prioritizes availability and partition tolerance with eventual consistency, leveraging techniques like consistent hashing, vector clocks, and Quorum technique.

Amazon DynamoDB

CORE CONCEPTS

Here are few of the core concepts used to implement Dynamo DB.

Consistent Hashing

Vector Clocks

Quorum Technique

CONSISTENT HASHING

- A technique to evenly distribute data across nodes in a network, minimizing data movement when nodes are added or removed.
- **Data Placement:** Nodes and keys are mapped onto a virtual circular ring using a hash function.
- **Minimal Data Movement:** Only keys near the affected node need to be redistributed.
- **Scalability:** Adding/removing nodes doesn't require rebalancing the entire system.

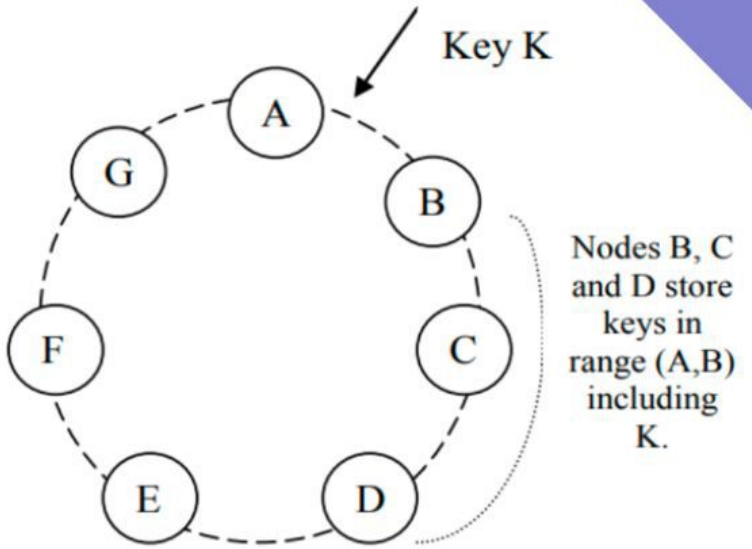


Fig. 1. **Partitioning and replication of keys in Dynamo ring.**

VECTOR CLOCKS

- A mechanism to track the version history of data in distributed systems, enabling conflict detection and resolution
- Each data item is assigned a vector clock, which is essentially a list of counters, one for each node in the system.
- **Conflict Detection**: Conflicting updates are identified when vector clocks indicate no causal relationship.
- **Reconciliation**: Applications or the system resolve conflicts by merging versions.

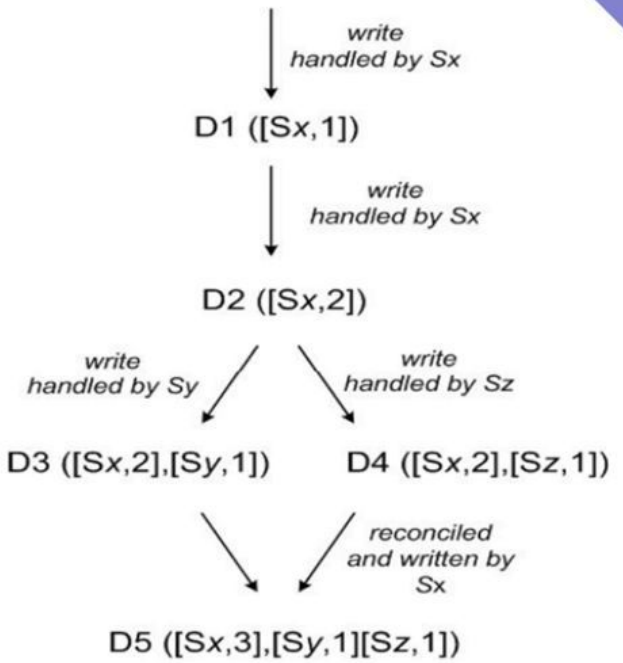
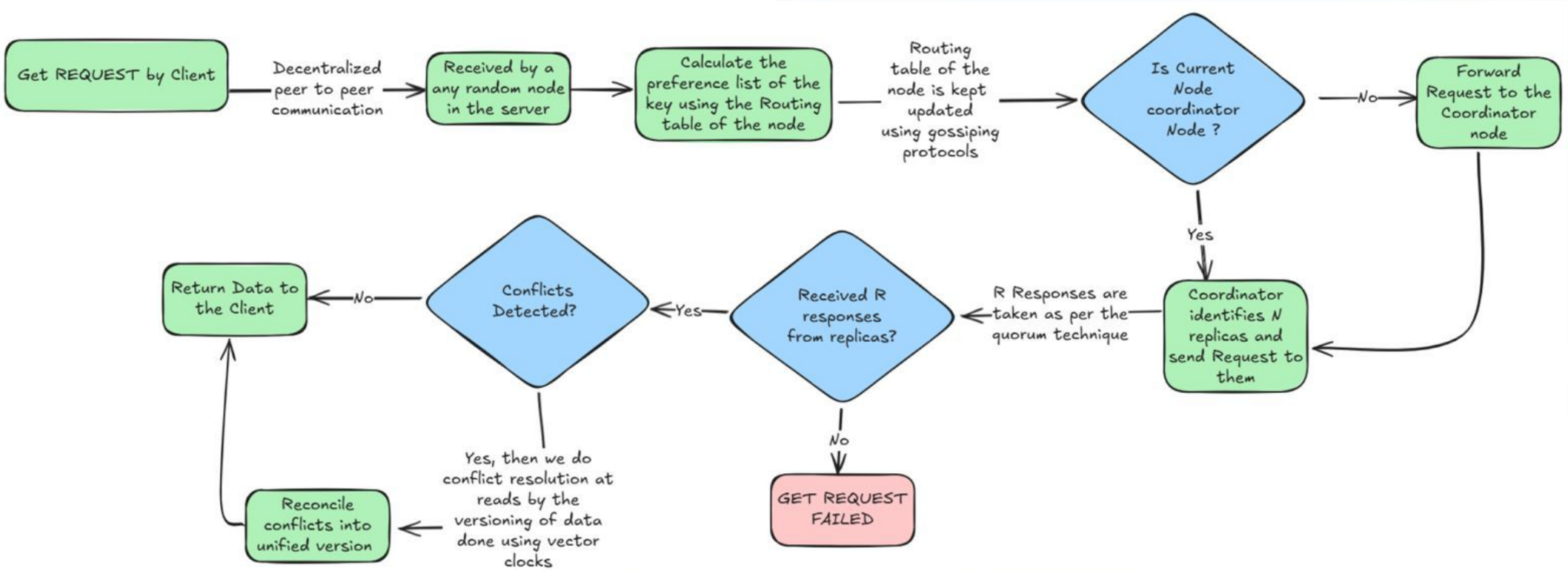


Fig. 2. **Version Evolution of an object over time.**

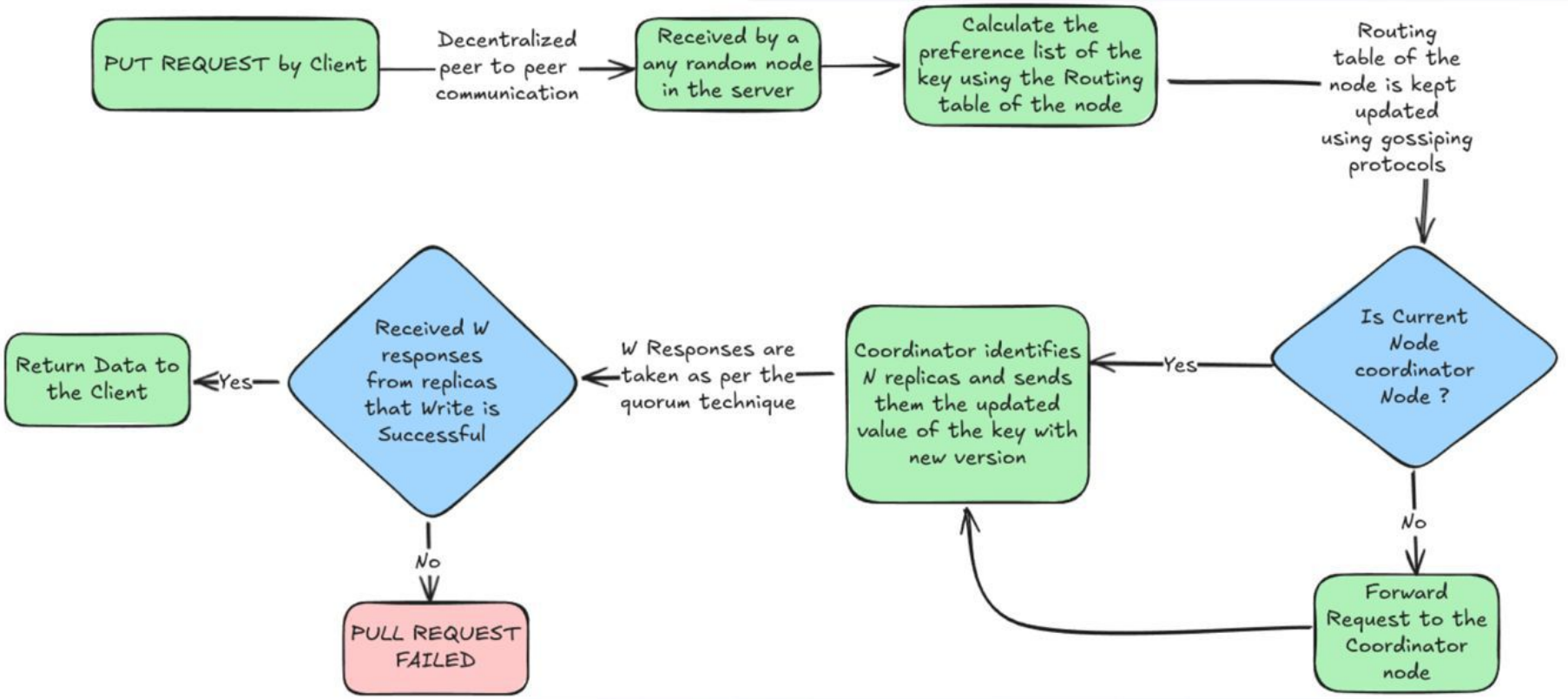
QUORUM TECHNIQUE

- A method to maintain consistency and reliability in distributed systems through partial agreement among nodes.
- **Read Quorum (R)**: Minimum number of nodes needed to respond to a read request.
- **Write Quorum (W)**: Minimum number of nodes needed to acknowledge a write request & the rest of the nodes get updated asynchronously.
- **Consistency Rule**: Ensures R+W>N (total replicas), so reads and writes overlap at atleast one common node.

GET OPERATION



PUT OPERATION



Asynchronous Operations

- We use daemon threads to implement these operations.
- These threads run in background and help to achieve eventual consistency.

Gossip Thread

Replica Sync Thread

Ping Thread

Asynchronous Operations

- Each thread executes specified function after certain time interval.
- **Gossip Thread**
  - Helps to sync the routing table.
- **Replica Sync Thread**
  - Helps to sync replicas.
- **Ping Thread**
  - Helps to detect failure of node. If a node does not respond the ping for a certain time interval, then it is assumed to be down and accordingly routing table is updated and a down routing table is maintained for down nodes.

Demo:

For the complete implementation we have used python programming language, RPyC for communication between servers & Redis as the persistent backend key-value store for storing each node's data.

Use Cases:

- **Shopping Cart**: Key-value pairs represent items and their quantities. Supports fault-tolerant, write-available services.

- **Instagram Post Likes**: Store like counts efficiently with real-time updates across replicas.

- **General Applicability**: DynamoMINI is a natural choice to all those systems requiring high availability, fault tolerance, low latency for reads and writes, and which allow for eventual consistency.

Conclusion & Future Scope

- Implemented a Simplified Dynamo: Leveraged consistent hashing for efficient data partitioning and replication.
- Ensured High Availability and Fault Tolerance: Used vector clocks, quorum techniques, and reconciliation algorithms for consistency and recovery.
- Achieved Robustness and Scalability: Integrated dynamic load balancing and decentralized routing through the gossip protocol.
- Delivered a Reliable Storage Solution: Developed a system capable of handling failures while maintaining eventual consistency.
- Future Scope: Some features like Merkle Trees for Efficient Reconciliation & Hinted Handoff for Enhanced Availability can be added to make the system more robust.

DynamoMINI

Thank You

29 November, 2024